

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине Построение и анализ алгоритмов

Студентка гр. 4343

Резяпова А.Р.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Задание

1. Реализовать алгоритм Ахо-Корасик для множественного точного поиска набора образцов в тексте.

2. Вариант 2. Подсчитать количество вершин в автомате; вывести список найденных образцов, имеющих пересечения с другими найденными образцами в строке поиска.

Описание алгоритма

1. Алгоритм Ахо-Корасик

Основная идея:

Алгоритм Ахо-Корасик позволяет найти все вхождения множества строк-образцов в текст за линейное время $O(|T| + \text{сумма длин образцов})$. Алгоритм состоит из трёх этапов: построение бора, построение суффиксных и конечных ссылок, поиск в тексте.

Шаг 1. Построение бора:

Бор (префиксное дерево) строится из всех образцов. Каждая вершина соответствует некоторому префиксу одного или нескольких образцов. Вершины, соответствующие целым образцам, отмечаются как терминальные с указанием номера образца.

Шаг 2. Построение автомата (суффиксные и конечные ссылки):

Для каждой вершины вычисляется суффиксная ссылка — вершина, соответствующая наибольшему собственному суффиксу строки текущей вершины. Суффиксные ссылки строятся с помощью BFS от корня. Конечная ссылка (output link, сжатая суффиксная ссылка) для вершины указывает на ближайшую терминальную вершину, достижимую по суффиксным ссылкам. Это позволяет при поиске не подниматься по всей цепочке суффиксных ссылок, а переходить сразу к следующему образцу за $O(1)$.

Шаг 3. Поиск в тексте:

Обход текста происходит по автомату.

На каждом шаге:

- Выполняется переход по текущему символу (с использованием суффиксных ссылок при отсутствии прямого перехода)
- Проверяется текущая вершина на наличие образца
- Проверяется цепочка конечных ссылок для нахождения всех образцов, оканчивающихся в текущей позиции

2. Индивидуализация (вариант 2)

Подсчёт количества вершин:

Количество вершин в автомате равно размеру массива вершин бора после завершения построения. Эта величина хранится в `nxt.size()`.

Поиск пересекающихся образцов:

Два вхождения пересекаются, если их интервалы $[pos, pos+len-1]$ имеют хотя бы один общий символ. Для определения пересекающихся образцов:

1. Собираются все найденные вхождения в виде (позиция, номер_образца, длина)
2. Вхождения сортируются по позиции
3. Для каждого вхождения проверяются последующие вхождения: если начало следующего $\leq$ конец текущего, образцы пересекаются
4. Номера пересекающихся образцов сохраняются в множество для устранения дубликатов

Оценка сложности:

Временная сложность:

- Построение бора: $O(\sum |P_i|)$, $\sum |P_i|$ - сумма длин образцов

- Построение суффиксных ссылок: $O(\sum |P_i| \times |\Sigma|)$, $\sum |P_i|$ - сумма длин образцов

- Поиск в тексте: $O(|T|)$

- Поиск пересекающихся образцов: $O(M \log M)$, где $M = O(|T| \times n)$
— количество найденных вхождений в худшем случае.

Общая сложность:

$O(\sum |P_i| \times |\Sigma| + |T| + |T| \times n \times \log(|T| \times n))$, $\sum |P_i|$ - сумма длин образцов

Сложность по памяти:

$O(\sum |P_i| \times |\Sigma|)$, где $|\Sigma| = 5$, $\sum |P_i|$ - сумма длин образцов

Структура данных для хранения автомата:

Для хранения автомата используется массив фиксированного размера для каждой вершины (`array<int,5>`). Такой выбор обусловлен малым размером алфавита (5 символов), что даёт:

- Доступ к переходу за $O(1)$

- Компактное хранение в памяти без накладных расходов на хеш-таблицы

Тестирование:

Входные данные	Вывод
ACGTAACGT 3 ACG GTA CGT	Результаты поиска 1 1 2 3 3 2 6 1 7 3 Пересекающиеся образцы: ACG GTA CGT
AAAAAA 3 AA AAA AAAA	Результаты поиска 1 1 1 2 1 3 2 1 2 2 2 3 3 1 3 2 3 3 4 1 4 2 5 1 Пересекающиеся образцы: AA AAA AAAA
AGACAGACAG 1 AGAC	Результаты поиска 1 1 5 1 Пересекающиеся образцы: нет
CAGCCAGCCAGC	Результаты поиска

4	1 4
AGC	2 1
AGCC	2 2
AGCCA	2 3
C	4 4
	5 4
	6 1
	6 2
	6 3
	8 4
	9 4
	10 1
	12 4
	Пересекающиеся образцы: AGC AGCC AGCCA C
AGCTAGCT	Результаты поиска
2	1 1
AGC	2 2
GCT	5 1
	6 2
	Пересекающиеся образцы: AGC GCT

Вывод

В ходе лабораторной работы реализован алгоритм Ахо-Корасик для множественного поиска образцов в тексте. Алгоритм позволяет найти все вхождения набора образцов в текст за один проход по тексту. Ключевым элементом алгоритма является построение автомата с суффиксными и конечными ссылками, что обеспечивает эффективный переход между состояниями при несовпадении и позволяет находить все образцы, оканчивающиеся в текущей позиции.

В рамках индивидуализации (вариант 2) выполнены следующие задачи: подсчитано количество вершин в построенном автомате и выведен список образцов, имеющих пересечения в строке поиска. Пересечением считается наличие хотя бы одного общего символа в интервалах вхождений образцов.